Technische Informatik Blatt 6

Einerkomplement:

$9 = 01001 \rightarrow -9 = 10110$

- Vorteil $0_{10} = 0000_1 = 1111_1$

Höchstwertiges Bit = 0 $\rightarrow \sum_i z_i \cdot 2^i$

Höchstwertiges Bit = 1 $\rightarrow -2^n + 1 + \sum_i z_i \cdot 2^i = (z_{n-1}, \ldots, z_1, z_0)_2$

Zweierkomplement

$9 = 01001 \rightarrow -9 = \begin{array}{r} 10110 \\ +\ \ 1 \\ \hline 10111_2 \end{array}$

Höchstwertiges Bit = 0 $\rightarrow \sum_i z_i 2^i$

Höchstwertiges Bit = 1

$\rightarrow -2^n + \sum_i z_i 2^i = (z_{n-1}, \ldots, z_1, z_0)_2$

Binäre Multiplikation:

Schulmethode: $0011 * 1010$

$$
\begin{array}{rl}
(+) & \left\{ \begin{array}{l} 0011 \\ 0000 \\ 0011 \\ 0000 \end{array} \right. \begin{array}{l} \cdot 1 \\ \cdot 0 \\ \cdot 1 \\ \cdot 0 \end{array} \\
\hline
= & 0011110
\end{array}
$$

Zweierkomplement (*):

$3 = 0011 \rightarrow -3 = 1101$

$2 = 0010$

$1101 \cdot 0010$

$$
\begin{array}{l}
0000 \\
0000 \\
1101 \\
\hline
11010
\end{array}
$$

Rest überlauf weg lassen

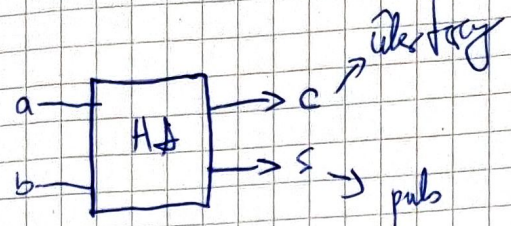
$-2^6 + 2^5 + 2^4 + 2^3 + 2 = -6$

Bei jeder 0 1 eine zusätzliche ein Byte

Technische Informatik

Halbaddierer

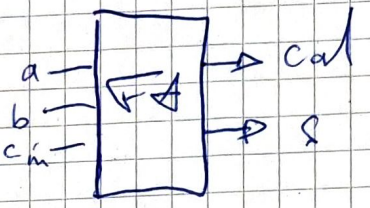
a	b	s	c
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



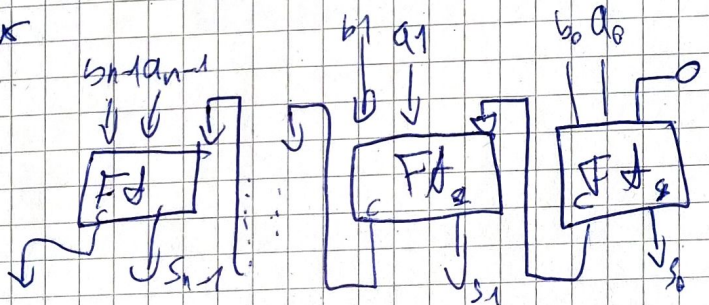
$c = a * b \quad , \quad s = a \oplus b$

Volladdierer

a	b	c_i	s	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



n-Bit Adder



Technische Informatik – Blatt 6

(25)

a) Zweierkomplement

i) $-91 + 89$

$91 : 2 = 45$ R1
$45 : 2 = 22$ R1
$22 : 2 = 11$ R0
$11 : 2 = 5$ R1
$5 : 2 = 2$ R1
$2 : 2 = 1$ R0
$1 : 2 = 0$ R1
$0 : 2 = 0$ R0 $\Rightarrow 91_{10} = 01011011_2$

-91 in Zweierk.:

$$\begin{array}{r} 10100100 \\ +\quad\ 1 \\ \hline 10100101 \end{array}$$

$89 : 2 = 44$ R1
$44 : 2 = 22$ R0
$22 : 2 = 11$ R0
$11 : 2 = 5$ R1
$5 : 2 = 2$ R1
$2 : 2 = 1$ R0
$1 : 2 = 0$ R1
$0 : 2 = 0$ R0 $\quad 89 = 01011001$

$$\begin{array}{r} 10100101 \\ 01011001 \\ \hline 11111110 \end{array} \Rightarrow -2^8 + 2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 0 = -2$$

ii) $59 : 2 = 29$ R1
$29 : 2 = 14$ R1
$14 : 2 = 7$ R0
$7 : 2 = 3$ R1
$3 : 2 = 1$ R1
$1 : 2 = 0$ R1
$0 : 2 = 0$ R0 $\Rightarrow 59_{10} = 0111011_2$

$12 : 2 = 6$ R0
$6 : 2 = 3$ R0
$3 : 2 = 1$ R1
$1 : 2 = 0$ R1
$0 : 2 = 0$ R0 $\Rightarrow 12_{10} = 00001100_2$

$\hookrightarrow -12_{10} = 11110011 + 1 = 11110100$

$$\begin{array}{r} 01110111 \\ \times\ 11110100 \\ \hline 00011100 \end{array}$$

$\hookrightarrow 2^3 + 2^2 + 2^1 + 2^0 = 47$

Technische Informatik – Blatt 6

(6-3)

g) m) $3 = \#011$

$74 = 0100\,1010 \rightarrow -74 = 1011\,0110$

$1011\,0110 * \#011$

$= -262$

↳ Das negative Vorzeichen muss natürlich beibehalten werden.

Technische Informatik Blatt 6

(6-3)

b) Führe die folgende Division aus

i) 91 % 13

91 = 01011011

13 = 1101

01011011 / 1101 = 0011

```
01011011 | 1101
-0000    |||| 0011
 1011    ||||
-0000    ||||
 10110   ||||
-1101    ||||
 10011   ||||
-1101    ||||
  01101  ||||
 -1101   ||||
  00011  ||||
    1101 ||||
    0000 ||||
```

ii) 59 = 111011

4 = 100

```
11101|11 / 100 = 1110
-100 |||
 101 |||
-100 |||
  011|||
 -000|||
  111|||
```

Eine Division mit einer Zweierpotenz entspricht, wie man hier, gut sieht, dem Verschieben des Teilers um die entsprechende Anzahl an Bits nach rechts

$59_{10} = 111011$ $\quad 4 = 2^2$

$111011_2 \,\%\, 4_{10} = 111011_2 \text{ shr } 2 = 1110 \text{ Rest } 11_2$

Technische Informatik – Blatt 6

(6-2)

c) Führen Sie folgende Division als vorzeichenloses Restoring-Division aus

i) 81 / 18

81 = 1010001 (a)

18 = 10010 (b)

q = 1010001 (a) 0000000

D = 18	A	Q = 81	
10010	0000000	1010001	// initialisiere
10010	0000001	0100010_	// ① left shift AQ
10010	1101111	0100010_	// A = A - D
10010	0000001	0100010	// A<0; restore A
10010	0000010	1000100_	// ② left shift AQ
10010	1110000	1000100_	// A = A - D
10010	0000010	1000100	// A<0, restore A
10010	0000101	0001000_	// ③ left shift AQ
10010	1110011	0001000_	// A = A - D
10010	0000101	0001000	// A<0, restore A
10010	0001010	0010000_	// ④ left shift AQ
10010	1111000	0010000_	// A = A - D
10010	0001000	0010000	// A<0, restore A
10010	0010100	0100000_	// ⑤ left shift AQ
10010	0000010	0100000_	// A = A - D
10010	0000010	0100001	// A>0
10010	0000100	1000001_	// ⑥ left shift AQ

Rechnerarchitektur – Blatt 6

6-2)

c)

10010	1110010	1000011	// A = A - D
10010	0000100	1000010	// A < 0, restore
10010	0001001	0000101	// ③ left shift AQ
10010	1110111	0000010	// A = A - D
10010	0001001	0001001	// A < 0, restore
10010			
10010			
10010			

↳ 81/18 = 4 Rest 9

Restoring-Division

① Eingabe: A = 0...0 (n = length(X)), Q = X, X/D = ?

for 1:n do
  left shift AQ
  A = A - D
  if A < 0
    restore A and add 0
  else add 1
end
A = Rest
Q = X/D

// Wiederherstellende Division

Technische Informatik - Blatt 6

(6-3)

c) ii) 39 = 100 111

16 = 10000

D = 16	A	Q = 39	
10000	000000	100111	// init
			// 6 Shift AQ
10000	000001	001111	
10000	110001	001111	// A = A - D
10000	000010	011110	// A < 0 restore
10000	000010	011110	// 2 Shift
10000	110010	011110	// A = A - D
10000	000101	111100	// A < 0 restore
10000	000101	111100	// 3 Shift
10000	110101	111100	// A = A - D
10000	001010	111000	// A < 0 restore
10000	001010	111000	// 4 Shift
10000	111010	110001	// A = A - D
10000	001101	100011	// A < 0 restore
10000	001101	100011	// 5
10000	111101	100011	// A = A - D
10000	001110	000111	// A > 0
10000	001110	000111	// 6
10000	111110	000111	// A = A - D
10000	001111	000110	

39 / 16 = 2 R 7

Technische Informatik – Blatt 6

6.3

d) Multiplikation ohne Vorzeichen Schulmethode

i) 1010 · 1101

```
  1010
  1010
  0000
 1111010
10000010   = 130_10
```

ii) 0100 . 1000

```
  0100
  0000
 00000
000000
0100000   =
```

iii) 1111 0000 * 0 1001011

```
  00000000
  11110000
 000000000
0000000000
 11110000
  00000000
  11110000
10001100101000000
```

Rest:

```
 1
 1
 1
 1
 0
```

```
  00001
 10001
 01100
```

Technische Informatik - Blatt 6

e) Booth-Algorithmus für Multiplikation in Zweierkomplement

i) $17 = 010001$
$9 = 001001$

a			
010001	· 001001		
000000	001001	0	// Initialisiere // ⓪
101111	001001	0	// Subtrahiere Shift
110111	100100	1	// ② Add a
001000	100100	1	// Shift
000100	010010	0	// ③ Shift
000010	001001	0	// ④ Sub a
100001	001001	0	// Shift
111000	100100	1	// ⑤ Add a
001001	100100	1	// Shift
000100	110010	0	// Shift ⑥ = n
000010	011001	0	

11 = Shift
00 = Shift
01 = Add a
10 = Subtrahiere a

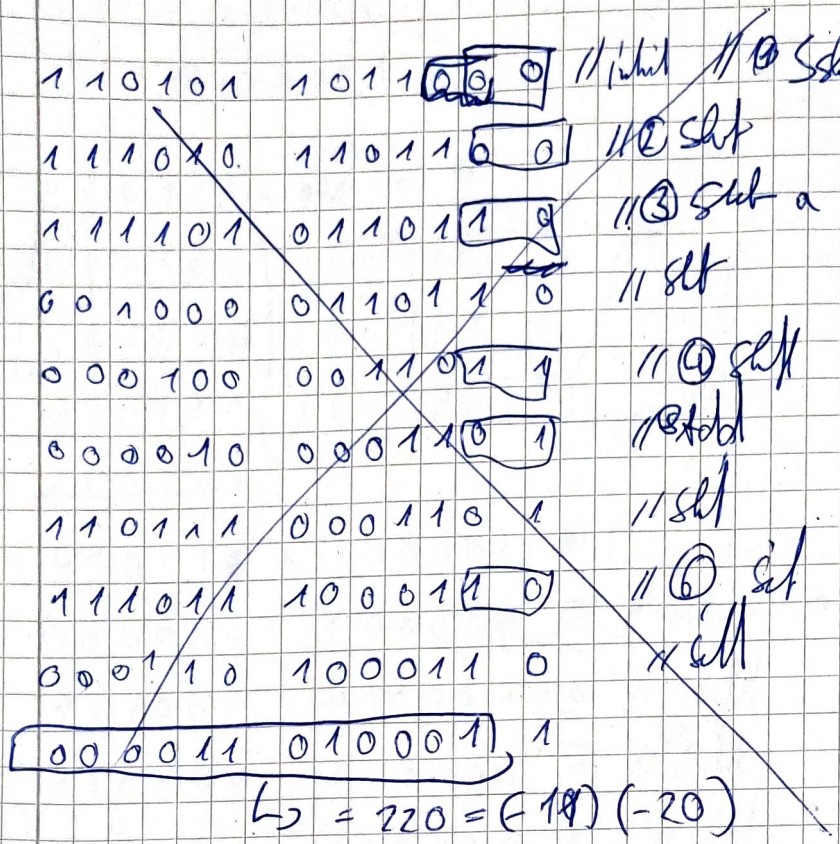
↳ $= 153 = 2^0 + 2^3 + 2^4 + 2^7$

Technische Informatik – Blatt 6

e) (ii) $(-11) \cdot (-20)$

$-11 = 110101$ $\quad n = 6$

$-20 = 101100$



$\hookrightarrow = 220 = (-11)(-20)$

Technische Informatik – Blatt 6

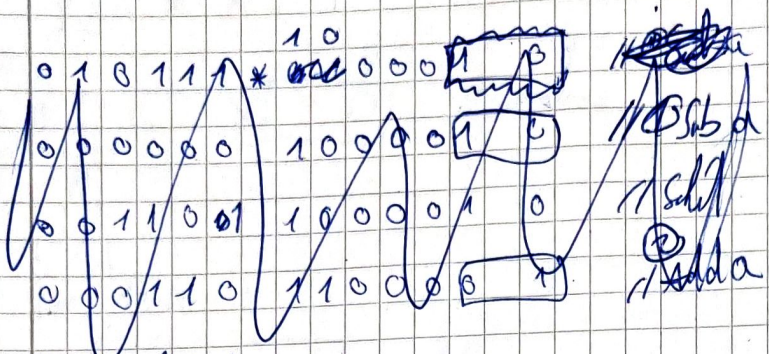
(A-3)

e) (11) 23 (-31)

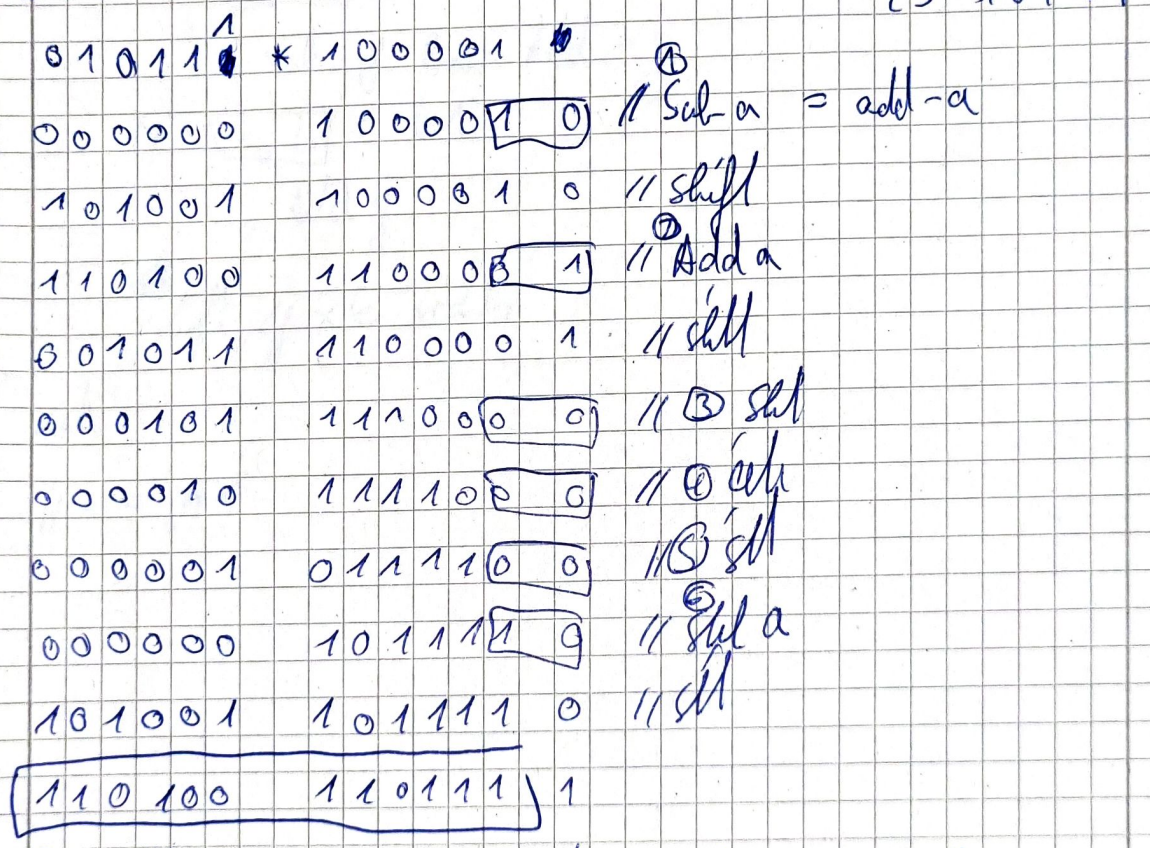
23 = 010111

-31 = 100001

n = 6



-23 = 101000 + 1 = 101001



$-2^{12} + 2^{11} + 2^{10} + 2^9 + 2^5 + 2^4 + 2^2 + 2^1 + 2^0 = -713$

Rechnerarchitektur – Blatt 6

Booth-Algorithmus für Multiplikation im Zweierkomplement

Gibt x_2 mit $\text{length}(y_2) = \log(x_2)$

$y_2 * x_2$

$0 \ldots 0$ (A) $\quad X \boxed{\quad 0}$ (b) // Initialize

For 1:4 do

if $\boxed{x_0\ b} = 10$

Sub y von Add $-y$

if $\boxed{x_0\ b} = 01$

Add y

Shift y x b rechts

end

$yx = y * x$

Technische Informatik – Übungsaufgabe

(5) (-32) · (-9) mit Booth Algorithm

b)

~~9 = 1001 → 9 = 0110~~

32 = 0100000 → -32 = 1011111 +1 = 1100000

9 = 001001 → -9 = 110110 +1 = 110111

n = 7

1100000 * 110111

0000000	110111 [1 0]	(1) Add -a
0100000	110111 0	Shift
0010000	011011 [1 1]	(2) Shift
0001000	001101 [1 1]	(3) Shift
0000100	000110 [1 1]	(4) Add a
1100100	000110 1	Shift
1110010	000011 [0 1]	(5) Add -a
0101001	000011 0	Shift
0010100	000001 [1 0]	(6) Shift
0001010	000000 [1 1]	~~Add a~~ Shift
0000010	010000	1

7 6 5 4 3 2 1 0

0 0 0 0 0 1 0 0 1 0 0 0 0

$2^7 + 2^4$

Technische Informatik - Blatt 7

7-1

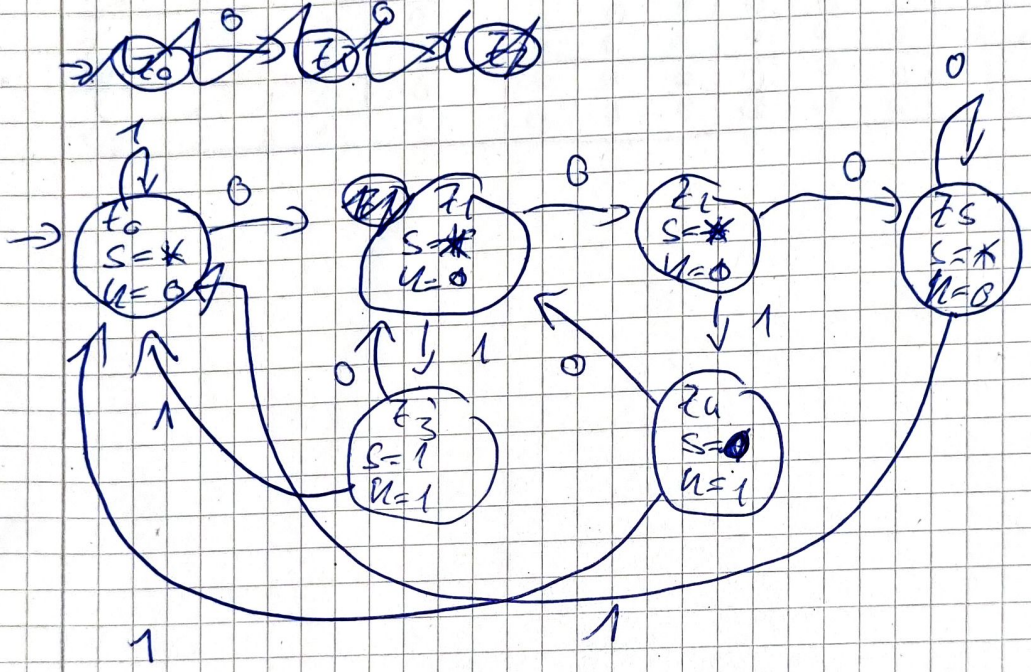
Banane 2 Takte ($x=0$) $\rightarrow S=0$ & $u=1$

Apfel 1 Takt ($x=0$) $\rightarrow S=1$ & $u=1$

Ei 1 Takte danach ($x=1$)

bei 2x 2< Takte $\rightarrow u=0$

a) Moore Automat



Technische Informatik – Blatt 7

(A-1)

c)

b)

	Zustand Q_2	Q_1	Q_0	Eingabe x	Folge Z Q_2	Q_1	Q_0	Ansteuerung D_2	D_1	D_0
Z_0	0	0	0	0	0	0	1	0	0	1
	0	0	0	1	0	0	0	0	0	0
Z_1	0	0	1	0	0	1	0	0	1	0
	0	0	1	1	0	1	1	0	1	1
Z_2	0	1	0	0	1	0	1	1	0	1
	0	1	0	1	1	0	0	1	0	0
Z_3	0	1	1	0	0	0	1	0	0	1
	0	1	1	1	0	0	0	0	0	0
Z_4	1	0	0	0	0	1	0	0	1	0
	1	0	0	1	0	0	0	0	0	0
Z_5	1	0	1	0	1	0	1	1	0	1
	1	0	1	1	0	0	0	0	0	0
Z_6	1	1	0	0	0	0	0	0	0	0
	1	1	0	1	0	0	0	0	0	0
Z_7	1	1	1	0	0	0	0	0	0	0
	1	1	1	1	0	0	0	0	0	0

} dann Hack

Technische Informatik – Übungsaufgaben

(a) 4 Sensoren:

→ Hall/Dubel-Sensor H

→ Lichtsensor L, genug Strom?

→ Drehsensor D, falls 360°

→ Glasbruchsensor G, Geschäftszeit

G=1 ⇒ Feldbesitzer F

Status-Lend Z „eingeschaltet"

L₀ selbe wie H=0

H=1 ⇒ in Zustand A gehen ⇒ H=1 ∧ L=1 → B

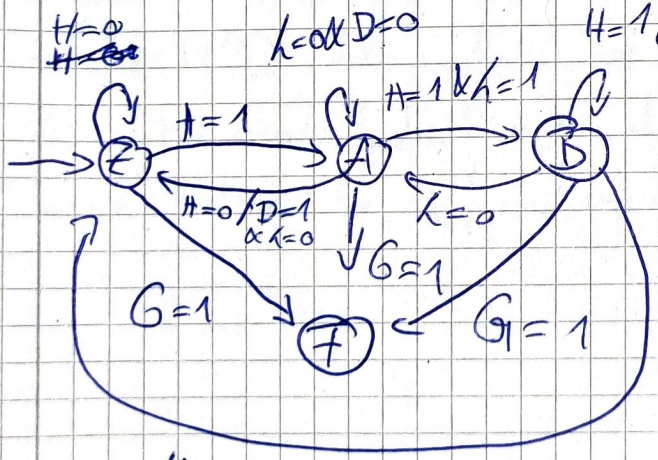
↻ L=0, D=0

Im A H=0 or D=1 ∧ L=0 ⇒ Z

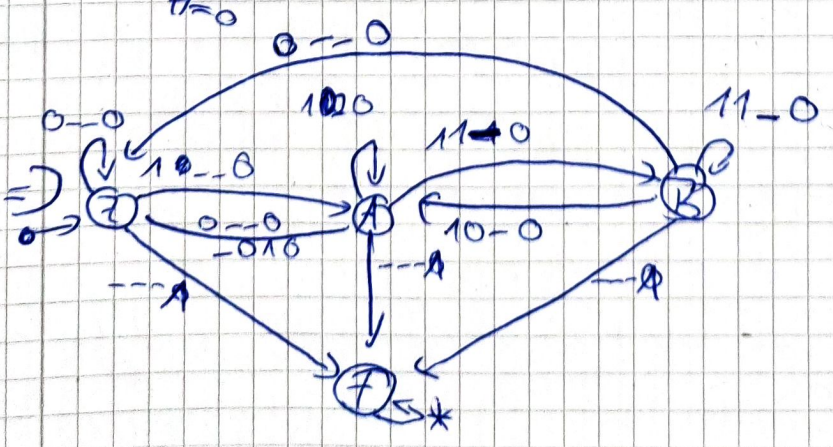
Schleife in B H=1 ∧ L=1

wenn L=0 → A ⟵ H=0 → Z

H L D G



Tafel Aufgabe!



Technische Informatik

Rechnerarchitektur

Stone: Organisation, Aufbau und Verbund von Komponenten eines Rechners. Entwurf spezieller und individueller Computer mit Hilfe einer mehr oder weniger umfangreichen Family von Standard-Komponenten.

Amdahl, Blaauw, Brooks: Rechnerarchitektur beschreibt Eigenschaften eines Systems, wie sie vom Programmierer wahrgenommen werden. Funktionales Verhalten und gedanklicher Aufbau eines Rechners, in Gegensatz zu dessen logischer und physikalischer Realisierung.

Unterschiede:

Amdahl: beschreibt externe Rechnerarchitektur, Stone die interne. Bei Amdahl beschreibt die Rechnerarchitektur Programmierschnittstellen mit Hilfe einer application program interface (API) oder instruction set architecture (ISA). Bei Stone beschreibt die Rechnerarchitektur die Mikroarchitektur, wie ein Rechner intern aufgebaut ist und seine Abläufe

Rechnerarchitektur

Rechnerarchitektur:

Nach Stone:

The study of computer architecture is the study of the organization and interconnection of components of computer systems. Computer architects construct computers from basic building blocks such as memories, arithmetic & busses.

From these building blocks the computer architect can construct anyone of a number of different types of computers, ranging from the smallest hand-held pocket calculator to the largest ultra-fast super computer.

In short, computer architecture is the discipline devoted to the design of highly specific and individual computers from a collection of common building blocks

Nach Amdahl, Blaauw, Brooks:

The term architecture is used here to describe the attributes of a system as seen by the programmer, i.e., the conceptual structure and functional behaviour, as distinct from the organization and data flow and control, the logical and physical implementation

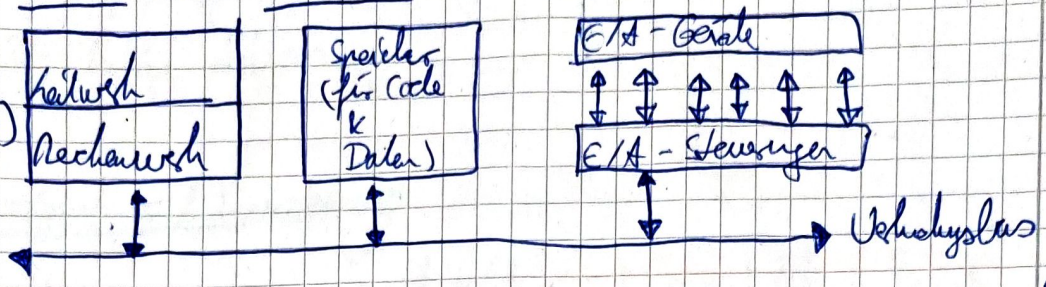
Externe Rechnerarchitektur

- Programmier - oder Befehlssatzschnittstelle
- Die Rechnermaschine

Technische Informatik

Von Neumann - Modell

1. Die Rechneranlage besteht aus den Funktionseinheiten Speicher, Leitwerk, Rechenwerk und Ein/Ausgabe-Einheit.



2. Die Struktur der Anlage ist weitgehend von bearbeiteten Problem. Die Anlage ist speicherprogrammierbar.
3. Anweisungen und Operanden werden in demselben physikalischen Speicher gespeichert
4. Der Speicher wird in Zellen gleicher Größe geteilt. Die Zellennummern heißen Adressen.
5. Das Programm besteht aus einer Folge von elementaren Befehlen, die in der Reihenfolge der Speicherung bearbeitet werden
6. Abweichungen von der Reihenfolge sind mit Sprungbefehlen möglich
7. Es werden Folgen von Binärzeichen verwendet, um alle Größen darzustellen
8. Kein explizites Angeben des repräsentierten Typs

Technische Informatik

Adressierungsarten

→ 0-stufige Adressierung (z.B. Operanden in Registern)
→ 1-stufige Speicheradressierung (z.B. klassische lw, sw-Befehle)
→ 2-stufige Adressierung

0-stufige Adressierung

→ Registeradressierung: ausschließlich Operanden aus dem Kern in Register

z.B. mfcr $15 // Reg[15] = cr
clr, 3 // D[3] = 0
ldpsw, 3 // D[3] = PSW

→ Unmittelbare Adressierung, Direktoperanden, immediate addressing:

z.B. lui $15, 3 // Reg[15] = 3 << 16
ld D3, #100 // D[3] = 100

1-stufige Adressierung

Genau 1 Zugriff auf den Speicher.

→ Direkte Adressierung, absolute Adressierung

z.B. lw $15, Adr // Reg[15] = Speicher[Adr]
ld D3, Adr // D[3] = Speicher[Adr]

→ Register-indirekte Adressierung

lw $15, ($2) // Reg[15] = Speicher[Reg[2]]
ld D3, (A4) // D[3] = Speicher[A[4]]

Technische Informatik

2-Stufige Adressierung

z.B: D[reg] = Speicher [Speicher [Adr]]

Adressmaschinen

3-Adress befehle Maschine:

IBM - Bereich „SSS" Befehle:

Speicher [S1] = Speicher [S2] ○ Speicher [S3]

z.B: add A1, a, b
mul d, A1, c

2-Adressmaschine

„SS" Befehle

Speicher [S1] = Speicher [S1] ○ Speicher [S2]

z.B: move A1, a // Speicher [A1] = Speicher [a]
add A1, b // Speicher [A1] = Speicher [b]
mult A1, c
move d, A1

1-1/2-Adressmaschine

Reg [r1] = Reg [r2] ○ Speicher [S]

Reg [r1] = Reg [r2]

z.B:
lw $8, a.
add $8, b
mul $8, c
sw $8, d

Technische Informatik

Adressmaschine

1-Adressmaschine

accu = accu o Speicher [S]

z.B.:
```
lw a      // accu = a
add b     // accu += b
mul c     // accu *= c
sw d      // d = accu
```

0-Adressmaschine (Stack)

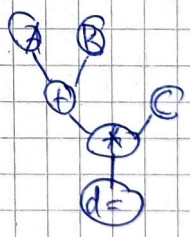
push a = Speicher [a] wird oberstes Stack-Element.

add = Addiert die beiden obersten Stack-Elemente, entfernt beide vom Stack, schiebt die Summe auf Stack

pop a = Speicher [a] = oberstes Stack-Element entfernt dies vom Stack

z.B.:
```
push a
push b
add
push c
mul
pop d
```

postfix Notation
AB+C*



Technische Informatik - Blatt 7

T 2)

a) $(a+b)^2 \cdot 10$ in verschiedenen Darstellungen

i)
```
lw   $8, a
add  $8, b
mul  $8, $8
muli $8, 10
sw   $8, a
```

1 1/2 - Adressmaschine
reg[1] = reg[1] + Speicher[5]

ii)
```
push a
push b
add
pop a
push a
push a
mul
push 10
mul
pop a
```

0 - Adressmaschine
Stack

iii)
```
lw   a
add  b
sw   a
lw   a
mul  a
mul  10
sw   a
```

1 - Adressmaschine
accu = accu + Speicher

iv)
```
add a, a, b
mul a, a, a
muli a, a, 10
```

3 - Adressmaschine
Speicher[s_1] = Speicher[s_2] + Speicher[s_3]

Technische Informatik - Blatt 7

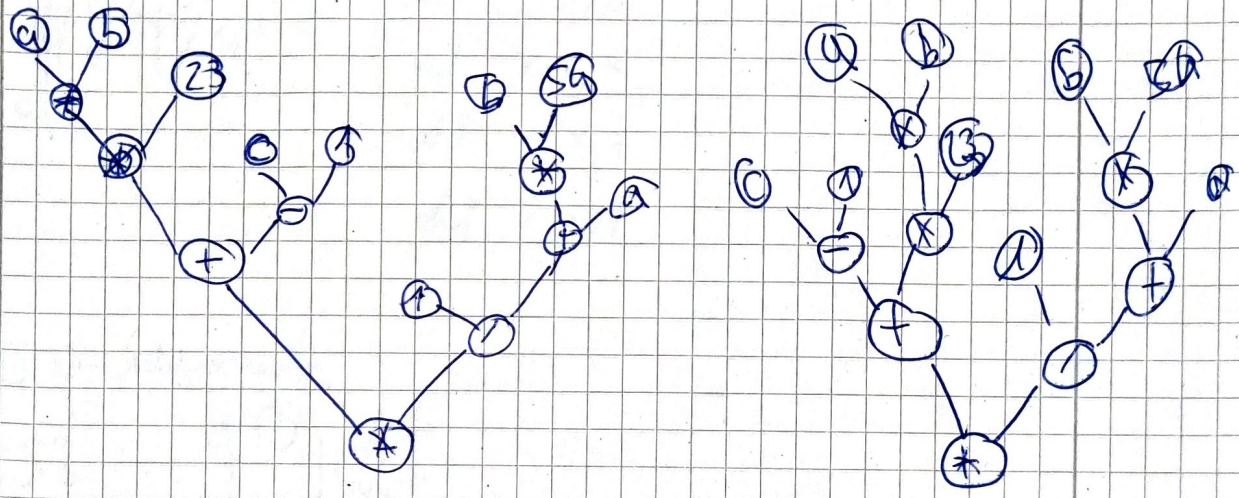
Was ist die UPN und was für eine Adressmaschine repräsentiert die UPN?

Umgekehrt polnische Notation für 0-Adressmaschine

(Z-2)

b) $((a \circ b \circ 23) + ((-1)) \circ \frac{1}{b \cdot 54 + a}$

ab 23 * * c1 - + 1 b54 * a + / *



UPN für 1/x => 1 x /

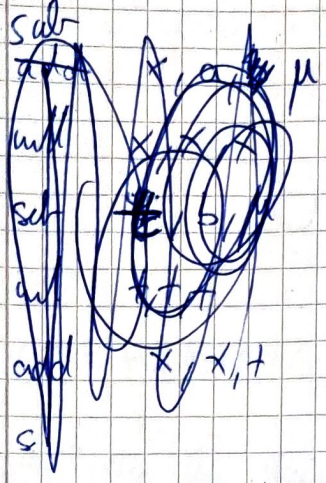
Technische Informatik Blatt 7

(Z-2)

a) $\sigma^2 = ((a-\mu)^2 + (b-\mu)^2) >> 1$

$\mu = (a+b) >> 1$

b) 3-Adressmaschine




```
add  x, a, b
shr  x, x, 1
sub  t, a, x
mul  t, t, t
sub  x, b, x
mul  x, x, x
add  x, x, t
shr  x, x, 1
```

c) 0-Adressmaschine

```
push a
push b
add
push 1
shr
pop  x
push a
push x
sub
pop  t
push b
push x
sub
pop  x
push x
mul
```

```
push t
push t
mul
push b
push x
sub
pop  t
```

```
push t
push t
mul
add
push 1
shr
pop  x
```

Technische Informatik – Blatt 8

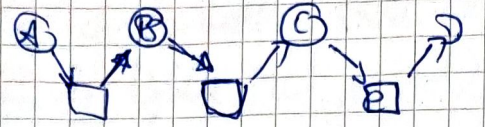
(8-1) Fließbandarbeit jeder Druck 2 Sekunden

A → B → C → D

a) Auswahl vor Lage bis D das Ergebnis hat, 975 Einträge

$t = 2 \cdot 4 \cdot 975 = 7800$ sek

b)



$t = 8 + 2(975 - 1) = 1956$ s

c) Wie lange dauert es bis d das erste Ergebnis da?

$t = 8$ s

d) Druck 6 sek

$t = (2+2+2+6) + 6 \cdot 974 = 5856$ s

Technische Informatik – Übungsaufgaben

6)

a) Sie haben in der Vorlesung eine ganze Reihe von Befehlsgruppen kennengelernt, in die sich Befehle einteilen lassen.

Befehlsgruppen:

- → Transferbefehle (lw, sw, mfhi)
- → E/A - Befehle (in, out)
- → Arithmetische Befehle (add, sub, mul, div)
- → Logische Befehle (and, or, not)
- → Vergleichsbefehle
- → Bitfeld- und Flag Befehle
- → Schiebebefehle
- → Sprungbefehle
- → Kontrollbefehle
- → Unmittelbarbezogene Befehle

Technische Informatik – Klausurvorbereitung

5) Adressierung:

n für n-stellige Adressierung, k für k-Adressmaschine

	n	k
accu = accu / Speicher [addr]	1	1
Reg [4] = Reg [2] XOR Speicher [Speicher [Reg [1]]]	2	3
Stack [top] = Stack [top] - Stack [top + 1]	0	0
Speicher [addr] = Reg [0]	1	1 - 1/2
Speicher [Speicher [Reg [3] + 2 * Reg [8] + 0x1C]] = Speicher [Speicher [Reg [3] + 2 * Reg [8] + 0x1C]] shl Speicher [addr]	2	2

3-Adressmaschine:

Speicher [S1] = Speicher [S2] ○ Speicher [S3]

2-Adressmaschine

Speicher [S1] = Speicher [S1] ○ Speicher [S2]

1-1/2-Adressmaschine

Reg [R1] = Reg [R1] ○ Speicher [S]

1-Adressmaschine

accu = accu ○ Speicher [S]

0-Adressmaschine

Stack

Technische Informatik – Übungsaufgaben

(6)

c) Welche Vorteile haben CISC-Befehlssätze und welche Eigenschaften haben CISC-Prozessoren in Bezug auf die Verarbeitung dieser Befehlssätze.

CISC

Vorteil: Komplexe Programme möglich, da Befehlssatz sehr mächtig ist.
Mehrere interne Zyklen werden pro CISC-Instruktion benötigt.
Mikroprogramm interpretiert Instruktion

=> Computer kennt viele Befehle gar nicht mehr

RISC:

Wenige, einfache Befehle wegen: => Feste Befehlswortlänge
- Hohe Taktfrequenzgeschwindigkeit
  ↳ Durch kleine Anzahl von Taktzyklen pro Befehl
  ↳ Durch Fließbandverarbeitung

Programlaufzeit = # Befehle * CPI-Wert * Dauer Zyklus

CPI-Wert = Ø Taktzyklen

RISC ist ohne Mikroprogrammierung realisierbar